Airton Luiz Canale Filho
João Pedro Ayache

Integração i-Educar → Moodle

Porto Alegre

2026

Airton Luiz Canale Filho
João Pedro Ayache

Integração i-Educar → Moodle

Relatório da Entrega 1 da disciplina Projeto e Arquitetura de Software, apresentando a implementação de uma integração monolítica entre o i-Educar e o Moodle.

Pontifícia Universidade Católica do Rio Grande do Sul – PUCRS

Escola Politécnica

Projeto e Arquitetura de Software

Professor: José Pedro Schardosim Simão

Porto Alegre

2026

RESUMO

Este relatório descreve a arquitetura e a implementação de uma integração monolítica entre o *i-Educar* e o *Moodle*, feita como aplicação Python modular. A solução sincroniza turmas e vínculos (matrículas e professorados) de forma síncrona e idempotente, usando o campo `idnumber` do Moodle como chave de correlação. Todo o ambiente é orquestrado em um único `docker-compose` que inclui o i-Educar, o Moodle e a aplicação de integração.

Palavras-chave: i-Educar. Moodle. Integração de sistemas. Monólito modular.

LISTA DE ILUSTRAÇÕES

Figura 1 – Componentes principais e dependências externas. 8
Figura 2 – Fluxo de sincronização completa (turmas + vínculos). 10
Figura 3 – Painel da integração com o histórico das execuções. 14

LISTA DE TABELAS

Tabela 1 – Portas e implementações. 9
Tabela 2 – Resumo das duas execuções consecutivas da sincronização completa. . 13

SUMÁRIO

1 Objetivo **6**
2 Informações sincronizadas **7**
3 Arquitetura **8**
3.1 Separação de responsabilidades 8
3.2 Interfaces 8
4 Diagrama de fluxo **10**
5 Decisões técnicas **11**
6 Testes realizados **12**
7 Resultados **13**
7.1 Evidências de execução 13
8 Limitações **15**
9 Repositório e execução **16**

Referências **17**

1 OBJETIVO

Integrar o sistema de gestão escolar *i-Educar* (Portabilis, s.d.) ao ambiente de aprendizagem *Moodle* (Moodle, s.d.), de modo que turmas cadastradas no i-Educar passem a existir como cursos no Moodle, com os alunos matriculados e os professores vinculados. A operação deve ser executada sob demanda via API HTTP e também periodicamente por agendador interno, preservando idempotência (nova execução não duplica dados).

2 INFORMAÇÕES SINCRONIZADAS

- **Turmas** do i-Educar → **Cursos** no Moodle (campo `idnumber = ieducar-turma-{id}`).
- **Pessoas vinculadas** (alunos matriculados e professores) → **Usuários** no Moodle, criados sob demanda (`idnumber = ieducar-pessoa-{id}`).
- **Vínculos** (matrícula ativa, professorado) → **Inscrições** no curso Moodle correspondente, com papel `student` ou `editingteacher`.
- Metadados de execução ficam no Postgres da integração em três tabelas: `MapeamentoExterno` (dedup), `ExecucaoSync` (auditoria) e `FalhaSync` (erros individuais).

3 ARQUITETURA

A solução segue o estilo *monólito modular*: um único deployable Python (FastAPI (RAMíREZ, s.d.) e SQLAlchemy (SQLAlchemy, s.d.)) organizado em domínios bem delimitados, com portas e adaptadores entre o núcleo de negócio e as infraestruturas externas (i-Educar e Moodle). A Figura 1 mostra os componentes principais.

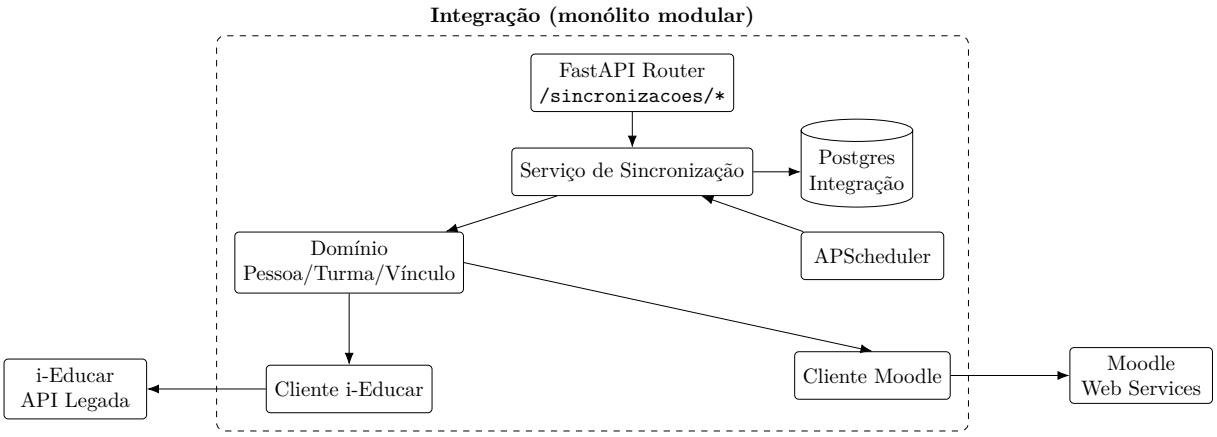


Figura 1 – Componentes principais e dependências externas.

3.1 SEPARAÇÃO DE RESPONSABILIDADES

`app/dominios/` Regras de negócio puras (modelos, serviços, portas). Não importa HTTP nem SQL. Define *o que* precisa acontecer.

`app/infraestrutura/` Adaptadores concretos: cliente HTTP para i-Educar (com tradutor de payloads para múltiplas versões), cliente REST para Moodle, e implementações *fake* em memória usadas em dev e testes.

`app/dominios/sincronizacao/` Orquestração: carrega dados do i-Educar, confere mapeamento no banco, chama Moodle quando necessário, registra execução e falhas.

`app/composicao.py` *Composition root* — escolhe entre cliente real e *fake* conforme `.env`.

`app/main.py` *Entrypoint* FastAPI, cria *schema* no boot e inicia o agendador.

3.2 INTERFACES

As portas de domínio são `Protocol`s Python (duck-typing estrutural). A Tabela 1 resume as principais.

Porta	Métodos	Implementações
`ClienteIEducar`	`listar_turmas, listar_vinculos, obter_pessoa`	`http`, `fake`
`ClienteMoodle`	`criar_curso, upsert_usuario, inscrever`	`http`, `fake`
`ExecucaoRepo`	`criar, finalizar, registrar_falha, listar, obter`	SQLAlchemy
`MapeamentoRepo`	`buscar, registrar`	SQLAlchemy

Tabela 1 – Portas e implementações.

4 DIAGRAMA DE FLUXO

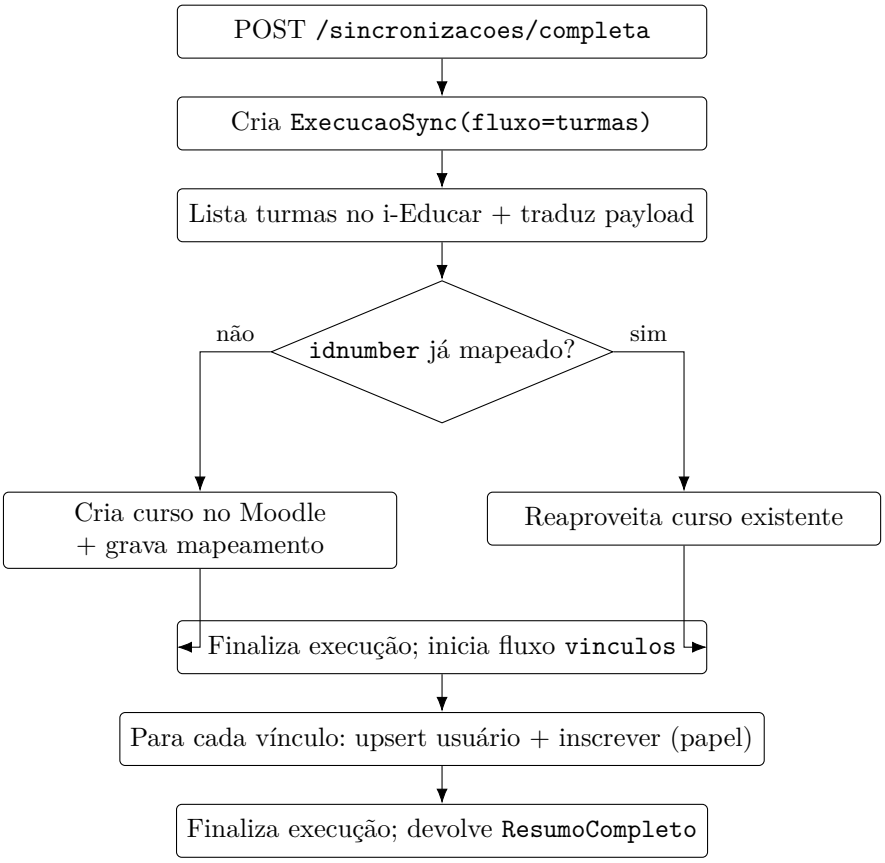


Figura 2 – Fluxo de sincronização completa (turmas + vínculos).

5 DECISÕES TÉCNICAS

Monólito modular (não microserviços). Escopo pequeno, deploy único, latência menor; a modularidade por portas mantém o custo de evolução baixo.

Sincronização síncrona com Postgres de controle. O enunciado exige fluxo síncrono. Três tabelas (`ExecucaoSync`, `FalhaSync`, `MapeamentoExterno`) bastam para auditoria e idempotência, sem introduzir fila.

`idnumber` prefixado como chave de correlação. Campo nativo do Moodle, não exige alterar schema do i-Educar. Prefixos (`ieducar-turma-`, `ieducar-pessoa-`) tornam a origem explícita.

Clientes *http* e *fake* atrás da mesma porta. O *fake* com dados seed habilita desenvolvimento sem subir i-Educar/Moodle e sustenta testes E2E determinísticos.

`httpx` com *retry/backoff* e classificação de erro. Erros transitórios dos dois sistemas são comuns; a camada HTTP separa falha fatal de retriável sem contaminar o domínio.

APScheduler *in-process*. Sem dependências externas (Redis, Celery). A limitação (uma instância só) é aceita conscientemente para o escopo da entrega.

Tradutor dedicado para payloads do i-Educar. A API legada varia nomes de campos entre versões; o tradutor isola essa normalização num ponto só.

`docker-compose` unificado via `include:`. Um único `docker compose up` sobe i-Educar, Moodle e a integração. O Moodle está versionado em `infra/moodle/`; o i-Educar oficial é obtido por `make infra` em uma versão fixada, sem copiar seu código para o repositório.

6 TESTES REALIZADOS

- **Unitários de domínio** (`tests/test_pessoa.py`): regras de nome, *username*, email sintético e `idnumber`.
- **Tradutor** (`tests/test_tradutor.py`): cobre variantes de campos de turmas e vínculos entre versões do i-Educar.
- **Orquestração E2E com *fakes*** (`tests/test_sincronizacao.py`): idempotência (segunda rodada não cria nada), inativação (remove inscrição), atribuição de papéis, e caminhos de falha gravando em `FalhaSync`.
- **Endpoints HTTP** (`tests/test_api.py`): via FastAPI `TestClient` com *dependency overrides* e `lru_cache` invalidado.

Comando: `pytest`. A suíte tem 24 testes, todos passando, e roda sem exigir rede ou containers externos.

7 RESULTADOS

A integração sincroniza turmas e vínculos de uma escola/ano a cada requisição a `POST /sincronizacoes/completa`, devolvendo um resumo com contagens de `processados`, `criados`, `reaproveitados` e `falhas`. Rodando duas vezes seguidas sobre a mesma base, a segunda execução reporta `criados = 0` e tudo em `reaproveitados`, confirmando idempotência via `MapeamentoExterno` e `idnumber`. O agendador interno repete o fluxo completo a cada `AGENDADOR_INTERVALO_MIN` minutos. Falhas individuais em pessoas/turmas não abortam a execução: ficam registradas em `FalhaSync` e o resumo marca o `status` apropriado.

7.1 EVIDÊNCIAS DE EXECUÇÃO

As evidências a seguir foram obtidas executando a aplicação real (FastAPI, serviço de sincronização e banco de controle) com os adaptadores *fake* do i-Educar e do Moodle (`IEDUCAR_CLIENTE=fake`, `MOODLE_CLIENTE=fake`), que simulam em memória duas turmas e quatro vínculos. As respostas da API foram salvas em `docs/evidencias/`. A Tabela 2 resume as duas execuções consecutivas de `POST /sincronizacoes/completa?ano=2025`.

Rodada	Fluxo	Processados	Criados	Reaproveitados	Falhas
1ª	turmas	2	2	0	0
1ª	vínculos	4	4	0	0
2ª	turmas	2	0	2	0
2ª	vínculos	4	0	4	0

Tabela 2 – Resumo das duas execuções consecutivas da sincronização completa.

A segunda rodada não cria nenhum registro novo e reaproveita todos os existentes, evidenciando a idempotência. A Figura 3 mostra o painel web da integração com o histórico dessas quatro execuções.

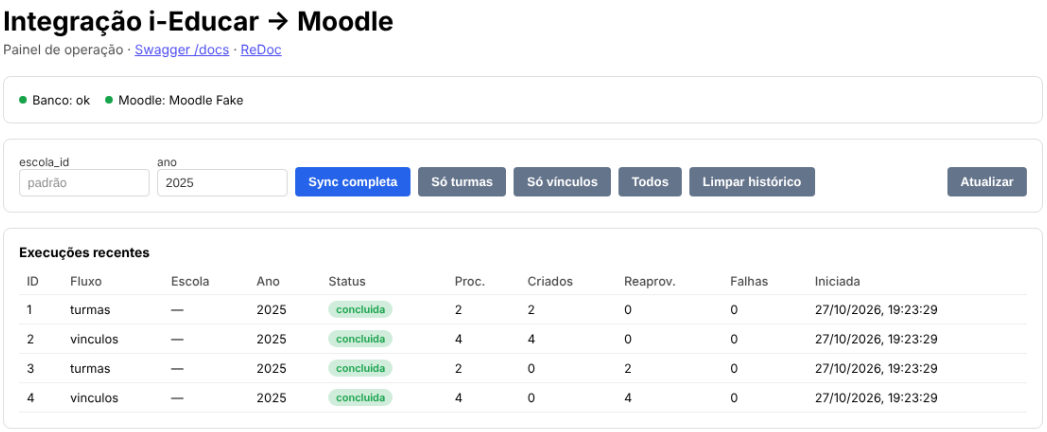


Figura 3 – Painel da integração com o histórico das execuções.

8 LIMITAÇÕES

- `FalhaSync` é persistida mas não há endpoint de reprocessamento automático — requer re-executar o fluxo completo.
- O *schema* é criado via `Base.metadata.create_all` no *boot*; não há Alembic nem versionamento de migrações.
- A API da integração não exige autenticação — assume execução em rede interna confiável (do *compose*).
- O agendador é *single-instance*; duas instâncias simultâneas rodariam o mesmo *job* em paralelo (sem *lock* distribuído).
- A exclusão lógica de turmas no i-Educar remove inscrições mas não arquiva o curso no Moodle.

9 REPOSITÓRIO E EXECUÇÃO

Instruções completas de *setup*, geração de credenciais (`IEDUCAR_ACCESS_KEY` e `MOODLE_TOKEN`) e demonstração estão no `README.md` do repositório. O caminho rápido é:

```
make infra
cp .env.example .env
docker compose up -d --build
make ieducar-install
# gerar tokens, colar em .env, docker compose restart integracao
curl -X POST "http://localhost:8000/sincronizacoes/completa?escola_id=1&ano=2025"
```

REFERÊNCIAS

Moodle. *Web services API*. s.d. Disponível em: <https://docs.moodle.org/dev/Web_services>. Acesso em: 27 out. 2026. Citado na página 6.

Portabilis. *i-Educar: software livre de gestão escolar*. s.d. Disponível em: <https://github.com/portabilis/i-educar>. Acesso em: 27 out. 2026. Citado na página 6.

RAMíREZ, S. *FastAPI*. s.d. Disponível em: <https://fastapi.tiangolo.com>. Acesso em: 27 out. 2026. Citado na página 8.

SQLAlchemy. *SQLAlchemy: the database toolkit for Python*. s.d. Disponível em: <https://www.sqlalchemy.org>. Acesso em: 27 out. 2026. Citado na página 8.